

D e p e n d e n c y I n j e c t i o n

Dependency Injection Pattern

Inject collaborators from outside

DEFINITION

Supply an object's dependencies from outside — injected at creation, via a setter, or through a container — instead of having it create them itself.

Instead of creating its collaborators, an object declares what it needs and lets something outside supply them — at creation time, through a setter, or via a container. That single inversion of "who builds my dependencies" is what makes an object swappable, mockable, and independent of how its collaborators are built.

WHY IT MATTERS

An object that news its dependencies is welded to concrete types: you can't substitute a fake in a test or a different implementation in production without editing it. Supplying dependencies from outside breaks that weld, pushes construction to one place (the composition root), and lets the same class run against real or test collaborators unchanged.

— How it works

Client

Declares the collaborators it needs (as parameters or injected fields) and uses them — without knowing their concrete types or how they're built.

Injector / Container

The wirer. Resolves abstractions to concrete impls and supplies them — a framework container (Nest, Angular, Spring) or just hand-written wiring.

Service

The provided dependency, referenced by the client through an abstraction so it can be swapped or faked.

HOW TO APPLY

1. Have a class depend on an interface and receive it from outside — never new it.
2. Bind the interface to a concrete impl once, at the composition root or module.
3. Inject required deps at creation; use a setter for optional ones.
4. In tests, just pass a fake — no container needed.

LITMUS TEST

Scan the class for new on a collaborator, or a global/singleton lookup. If it builds or fetches its own dependencies, they aren't injected.

— Smell → fix

A service that builds its own dependency — now welded to `SqlOrderRepo`:

```
class OrderService {  
  private repo = new SqlOrderRepo(db) // self-wired  
  place(o: Order) { this.repo.save(o) }  
}  
// welded to SqlOrderRepo; tests need a real database
```

↓ INVERT THE WIRING

```
@Injectable()  
class OrderService {  
  @Inject(OrderRepo) private repo!: OrderRepo // field-injected  
  place(o: Order) { this.repo.save(o) }  
}  
// module: { provide: OrderRepo, useClass: SqlOrderRepo }
```

`OrderService` now declares `OrderRepo` and the container supplies the concrete `SqlOrderRepo`, wired once in the module. Tests assign a fake repo and need no database. DI is the technique; a container just automates wiring you could do by hand with a factory at the composition root.

USE WHEN

- ✓ Realizing DIP
- ✓ Testability via mocks; swapping implementations or config

AVOID WHEN

- ▲ Tiny scripts
- ▲ Over-configured containers for trivial graphs

— Trade-offs & pitfalls

PROS

- + Testable, decoupled
- + Centralized wiring and lifecycles

CONS

- Indirection
- Container "magic"
- Startup wiring complexity

COMMON PITFALLS

- ▲ A container so configured it becomes its own untraceable framework.
- ▲ Service-locator disguised as DI: pulling deps from a global instead of receiving them.
- ▲ Over-injecting trivial graphs where a plain new at the edge would do.

WHERE YOU SEE IT

- NestJS / Angular / Spring containers resolving providers automatically.
- Pure DI: a factory function makeService(dep) wired at main().
- Passing a fake repository into a service under test.

SEE ALSO

DIP	DI is how you realize Dependency Inversion at runtime — depend on abstractions, inject the concretes
Repository	inject the repository interface so storage stays swappable and fakeable
Singleton	a DI container gives single, shared instances without the Singleton anti-pattern
Strategy	injecting which implementation to use is Strategy chosen by configuration

— Interview & sources

Is DI the same as a DI container?

No. DI is the technique — pass dependencies in from outside. A container (Nest, Spring) is optional tooling that automates the wiring; you can do DI by hand with a factory at the composition root.

Which injection style do you prefer, and why?

For required dependencies, inject them at creation: the object is never half-built and its needs are explicit. Setters suit optional ones; field injection hides dependencies and hurts testability.

DI vs the Dependency Inversion Principle?

DIP is the design rule — depend on abstractions, not concretes. DI is a technique that realizes it at runtime by supplying those abstractions from outside. A container is optional tooling on top.

Does DI remove the new keyword?

No — something must still create objects. DI pushes those new calls to the composition root (the container or main wiring), so the rest of the code depends on abstractions and stays testable.

REMEMBER

Don't let an object new its own collaborators — have them handed in.

SOURCES

Martin Fowler — "Inversion of Control Containers and the Dependency Injection pattern" (2004)

Mark Seemann — "Dependency Injection Principles, Practices, and Patterns" (2019)

NestJS, Angular, Spring — DI containers in modern frameworks